TrafficAlign: Aligning Large Language Models for Traffic Scenario Generation

Zhi Tu Liangkun Niu Tianyi Zhang
Purdue University
{tu85, niu61, tianyi}@purdue.edu
https://github.com/TrafficComposer/TrafficAlign

arXiv:2606.29097v1 [cs.CV] 27 Jun 2026

Abstract

Recent research has investigated the use of large language models (LLMs) to generate traffic scenarios for autonomous driving. However, pretrained LLMs often fail to align with real-world traffic distributions. In this work, we present TRAFFICALIGN*, an automated framework that synthesizes traffic scenarios based on real-world driving videos, performs data validation, and aligns LLMs with the synthesized scenarios. The evaluation shows that traffic scenarios generated by* TRAFFICALIGN *are highly effective, revealing up to 10.8% more collisions on average across three autonomous driving models than state-of-the-art methods. Furthermore, fine-tuning these driving models with* TRAFFICALIGN*-generated scenarios significantly reduced collision rates by 36.1% compared with the original models. A qualitative study using traffic datasets from six geographically diverse regions shows that* TRAFFICALIGN*-generated scenarios exhibit strong alignment with corresponding traffic distributions in these regions.*

1. Introduction

Despite significant progress in autonomous driving, traffic accidents caused by autonomous vehicles continue to occur [32]. This highlights the need for comprehensively testing driving models. In recent years, simulation-based testing has emerged as a cost-effective and scalable alternative method to real-world road tests. However, a critical challenge in simulation-based testing lies in effectively generating realistic traffic scenarios.

Recent studies have explored the use of large language models (LLMs) to automatically generate traffic scenarios [3, 10, 15, 44, 47, 48, 52]. However, most of these approaches require humans to write natural language descriptions of desired traffic scenarios in order to generate the simulation scripts. As a result, this manual input could become labor-intensive in large-scale testing. One exception is ChatScene [66], which reduces the dependency on manual effort by directly prompting LLMs to generate natural language descriptions of traffic scenarios. However, LLMs lack sufficient traffic domain knowledge and also tend to generate homogeneous scenarios. As a result, they struggle to generate scenarios that align with real-world traffic characteristics and distributions, especially for specific geographic contexts such as large cities, small towns, and mountainous regions.

To address these challenges, we propose TRAFFICALIGN, an automatic framework that synthesizes traffic scenarios from real-world driving videos, checks data validity, and aligns LLMs with the synthesized scenarios. To effectively validate the synthesized scenarios, TRAFFICALIGN converts the natural language description of each scenario into a formal representation in a domain-specific language (DSL). Compared with natural language, this DSL representation enables systematic analysis and rigorous checking. The identified issues are then fed back to the data synthesizer for refinement or regeneration. Finally, TRAFFICALIGN uses the validated scenarios to fine-tune LLMs so that it can generate realistic scenarios that align with the real-world traffic distributions of a target region.

We conduct a comprehensive evaluation by comparing TRAFFICALIGN with ChatScene [66], two adversary-based baselines [12, 54], two rule-based baselines [49, 68], and five LLM baselines. We first evaluate the testing effectiveness of traffic scenarios generated by each method on three driving models using the SafeBench platform [60]. TRAFFICALIGN reveals up to 10.8% more collisions across the three driving models than the strongest baseline (Section 4.2). We further demonstrate that these generated scenarios can be used to improve the safety of driving models. Specifically, the driving models fine-tuned with TRAFFICALIGN-generated scenarios reduce collision rate by 36.1% compared with the original models before fine-tuning when evaluated on a held-out set (Section 4.3). In addition, a qualitative study across geographically diverse regions shows that scenarios generated by TRAFFICALIGN align more closely with the real-world traffic distributions (Section 4.4).

Our main contributions are concluded as follows:

- We propose TRAFFICALIGN, an automated framework that synthesizes traffic scenarios from real-world driv-

ing videos, validates their quality, and aligns LLMs with these scenarios.
- TRAFFICALIGN-generated scenarios show high effectiveness in testing three autonomous driving models, revealing a collision rate up to 10.8% higher than the state-of-the-art baseline, ChatScene [66].
- Fine-tuning the driving models with TRAFFICALIGN-generated scenarios reduces the average collision rate by up to 36.1% compared with the original models, indicating TRAFFICALIGN's effectiveness in improving driving model performance.

2. Related Work

Traffic scenario generation. There is a large body of literature on traffic scenario generation. Prior work can be broadly classified into three categories. First, *search-based approaches* start from seed scenarios and explore the scenario search space through scenario mutation guided by adversarial learning [2, 26, 67] or heuristic functions [1, 7, 23, 27, 29, 45]. While effective in generating safety-critical scenarios, these approaches often incur a high computational cost and suffer from limited scenario diversity. Second, *scenario reconstruction approaches* [13, 18, 41, 52, 53, 63] generate traffic scenarios from real-world driving data, such as sensor logs and crash reports. These approaches generate realistic scenarios but suffer from data scarcity. Third, *rule-based approaches* [5, 25, 40, 51] encode traffic laws and physical constraints to compose scenarios. While they provide systematic and interpretable approaches for scenario generation, they require extensive manual rule encoding, which significantly limits their scalability. Moreover, since handcrafted rule sets rarely capture the full spectrum of traffic scenarios, their coverage of safety-critical cases is limited.

Another line of research learns a generative model of traffic scenarios [8, 11, 22, 36, 39, 46]. Meta-Sim [22] learns a probabilistic grammar and optimizes its parameters through differentiable rendering to minimize the domain gap between synthetic and real data, enabling adaptive generation of realistic training scenarios. STRIVE [39] learns a latent generative model of traffic scenarios and performs adversarial optimization to create realistic yet collision-prone scenarios for stress-testing planners. SLEDGE [8] leverages diffusion transformers to generate scalable, map-conditioned driving environments that capture diverse traffic configurations with controllable density. Scenario Diffusion [36] adapts diffusion models for controllable, structured driving scenario generation with constraints such as collision likelihood or traffic flow density. These methods model scenarios at the structured bird's-eye-view or trajectory level (e.g., lane graphs, boxes, trajectories), whereas TRAFFICALIGN evaluates scenarios as 3D traffic simulations in CARLA. We therefore do not include them as baselines in our evaluation.

Recent studies have explored the use of LLMs for traffic scenario generation [10, 15, 19, 47, 52, 66]. TARGET [10] uses an LLM (GPT-3.5) to generate traffic scenarios from traffic rules in driving handbooks. However, the generation of simulation scripts relies on human-curated rule-based algorithms. ScenicNL [15] addresses this challenge by combining a comprehensive prompting strategy with a compiler-in-the-loop to generate Scenic [16] scripts from crash reports using LLMs. However, it depends on human-written crash reports, which are labor-intensive and hard to scale up for massive traffic scenario generation. ChatScene [66] addresses this limitation by leveraging an LLM's internal knowledge and directly prompting it to design safety-critical scenarios, thereby reducing dependence on manually crafted scenario descriptions. However, LLMs' generalist knowledge lacks fine-grained, location-specific traffic understanding, causing the generated scenarios to deviate from real-world traffic distributions. TRAFFICALIGN addresses this issue by aligning an LLM's general knowledge with real-world observations. TRAFFICALIGN automatically synthesizes scenarios from large-scale real-world driving videos and aligns the LLM with the representations, thereby generating traffic scenarios consistent with real-world traffic distributions.

LLM alignment. While there are various methods for LLM alignment, the most related to us are methods that generate high-quality synthetic data for alignment [21, 24, 28, 55–57, 59, 61, 65]. Self-Instruct [55] proposes a semi-automated data synthesis pipeline that prompts a pretrained LLM to create diverse instruction–input–output triples and then applies heuristic validity and duplication filtering to generate high-quality and diverse data for LLM instruction-tuning. Evol-Instruct [61] introduces an evolutionary data synthesis method that iteratively rewrites instructions from a seed instruction pool into more diverse and complex instructions with failure filtering. CodecLM [57] proposes an LLM-guided instruction generator that maps seed tasks into metadata and decodes them into diverse instruction examples for fine-tuning. FLAME [28] proposes a factuality-aware data curation pipeline that classifies prompts by factual requirements and uses the LLM itself to generate factual responses, constructing hallucination-resistant fine-tuning data. In a similar spirit, TRAFFICALIGN automatically extracts traffic scenarios from videos and uses the synthesized scenario descriptions to align LLMs. In addition, TRAFFICALIGN introduces an effective validation method to analyze the semantics of the synthesized data (Section 3.2), improving the data quality for LLM alignment.

3. Method

In this work, we introduce TRAFFICALIGN, an automated framework that synthesizes traffic scenario representations

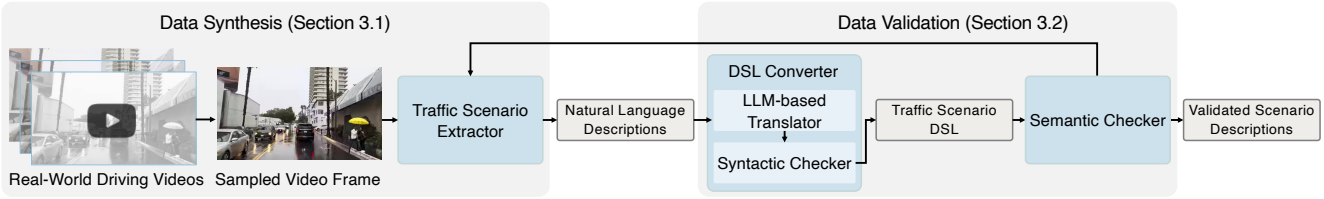


Figure 1. Overview of TRAFFICALIGN's data synthesis pipeline.

from real-world driving videos, checks data validity, and aligns LLMs with these representations. Figure 1 shows TRAFFICALIGN's data synthesis pipeline. TRAFFICALIGN first synthesizes traffic scenario descriptions from driving videos using a multimodal LLM (Section 3.1). To validate data quality, TRAFFICALIGN translates synthesized scenario descriptions into a formal representation in a domain-specific language (DSL) and performs systematic checks to detect semantically incomplete or irrelevant scenarios (Section 3.2). Using the synthesized scenario descriptions, TRAFFICALIGN aligns an LLM with the captured traffic distributions (Section 3.3).

3.1. Data Synthesis

To generate real-world aligned scenarios, TRAFFICALIGN first captures the real-world traffic distributions in the target region. We use driving videos as the data source because they are widely available online and cost-effective to collect. From these videos, TRAFFICALIGN automatically curates traffic scenario representations that reflect the underlying real-world traffic distributions.

Video Collection and Preprocessing. To demonstrate the generalizability of TRAFFICALIGN, we collect six video datasets covering geographically diverse regions and use TRAFFICALIGN to generate six synthetic datasets for alignment. These six regions include two metropolitan areas (Los Angeles and New York City), two national parks (Yosemite and Yellowstone), a small town in Pennsylvania, United States, and a country-scale dataset from Switzerland encompassing diverse driving environments. In total, we collect 261 first-person perspective scenic driving videos from YouTube [64]. TRAFFICALIGN uniformly samples frames from each video at one frame every 15 seconds (fps=$1/15$) to capture diverse traffic scenarios while minimizing near-duplicate frames. In-the-wild videos often contain irrelevant frames, such as titles, introductory screens, or auxiliary screens, which can result in invalid traffic scenario extraction. Scenarios synthesized based on these irrelevant frames will be systematically filtered by the semantic checkers in a later stage, as detailed in Section 3.2.

Traffic Scenario Description Synthesis. For each selected frame, TRAFFICALIGN employs a multimodal LLM to extract information about the depicted traffic scenario and synthesize a description in natural language. We use GPT-4.1 nano (`gpt-4.1-nano-2025-04-14`) [34] as the default model in TRAFFICALIGN. Since LLM output quality is highly sensitive to prompt design, we carefully construct a compositional prompt that integrates several effective prompting strategies, including role-playing [43], step-by-step instructions, chain-of-thought reasoning [58], and in-context few-shot learning [6]. The prompt includes four sections. First, the LLM is assigned the role of a traffic domain expert. Second, the task is introduced with step-by-step instructions using a chain-of-thought strategy. Third, we specify the output schema, which includes an intermediate reasoning trace and the natural language description of the traffic scenario. Finally, the prompt includes an in-context example that familiarizes the LLM with both the input image and the expected output format. Details on prompt design are available in Supplementary Material.

Specifically, the multimodal LLM is instructed to describe a traffic scenario with three parts. First, it needs to describe the road network and environment, including area type (urban or rural), road type (e.g., intersection, roundabout), directionality (one-way or two-way), number of lanes, weather, and time of day. Then, it should provide contextual details, such as traffic density, roadside context, and the presence of emergency vehicles or road construction. Finally, it should also describe the actors, including the ego vehicle and the surrounding non-player character (NPC) vehicles and pedestrians. The LLM is instructed to describe the position and behavior of each actor. Examples of natural language descriptions are provided in Supplementary Material.

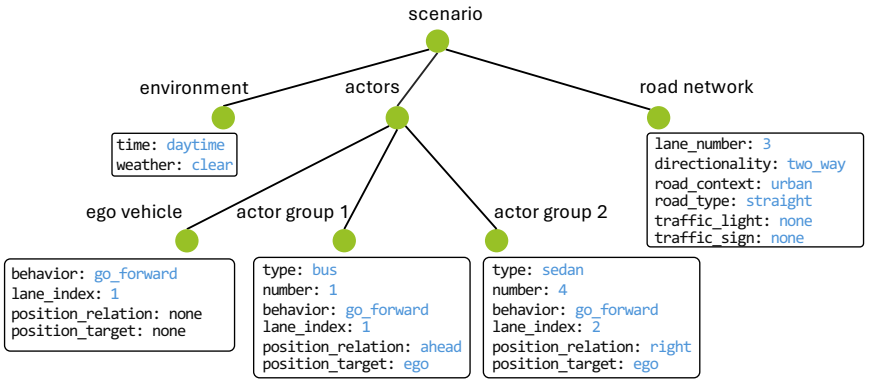
3.2. Data Validation

During the data synthesis stage, LLM hallucinations may produce semantically incomplete scenario representations. In addition, in-the-wild input videos often contain irrelevant frames, such as titles, introductory screens, or auxiliary screens, which result in invalid scenario representations. To address these potential issues and improve data quality for LLM alignment, TRAFFICALIGN performs semantic validation on the synthesized traffic scenario descriptions.

A symbolic DSL as proxy. Directly analyzing the semantics of the natural language descriptions is challenging and unreliable because such NL descriptions are unstructured and sometimes ambiguous. We therefore translate each de-

The traffic scenario occurs on a multi-lane urban arterial street with three lanes in total. The direction of travel for the ego vehicle includes two lanes and the opposite direction has one lane. It is in the daytime and is sunny with clear skies... No traffic lights are visible, nor any traffic signs... The road runs through a residential area, with buildings... Traffic flow is moderate; flow appears steady... The ego vehicle is driving forward in the second lane from right. Directly ahead, a bus is going straight... There are 4 sedans in the adjacent right lane...

(a) Natural language description.



(b) Symbolic representation using the domain-specific language [10].

Figure 2. Example natural language description of a traffic scenario and its translated domain-specific language representation for data validation. The natural language description is simplified for visualization.

scription into a formal representation in a domain-specific language (DSL) for systematic analysis and rigorous checking. We adopt an existing DSL design [10], since it provides a concise syntax grammar with sufficient coverage of various elements in a traffic scenario. Details of this DSL are provided in Supplementary Material.

TRAFFICALIGN translates natural language descriptions into formal DSL representations using GPT-5 (`gpt-5-2025-08-07`) [35]. The LLM prompt starts by configuring the LLM as a traffic domain expert and instructs it to identify the compositional elements of the traffic scenario step by step. It then presents the definition of the DSL grammar. The prompt ends with two in-context examples, each pairing a natural language description with its corresponding DSL representation. The DSL's context-free grammar requires exact symbol usage. TRAFFICALIGN ensures that the translated DSL representation is well-formed by checking its syntax correctness based on the DSL grammar. For example, if it detects an out-of-vocabulary symbol, TRAFFICALIGN will prompt the LLM again with the syntax error message and ask the LLM to refine it based on the grammar. Figure 2 illustrates a natural language description of a traffic scenario and its corresponding DSL representation.

Semantic validation. Using the formal DSL representation, TRAFFICALIGN systematically validates each scenario representation for semantic completeness and validity. For each DSL element, TRAFFICALIGN verifies that the essential attributes are specified. When multiple essential elements are missing (e.g., time of day and weather), this usually indicates that the source frame conveys little information related to traffic scenarios, so the *semantic checker* marks it as a non-scenario frame and discards it. If partial attributes of certain elements are missing (e.g., unspecified locations or behaviors for actors), the *semantic checker* marks the scenario description as incomplete and initiates a self-refinement loop.

Self-refinement. TRAFFICALIGN closes the loop between data synthesis and validation with a lightweight self-refinement step. When a scenario is identified as valid but semantically incomplete, TRAFFICALIGN appends the diagnostic messages to the original prompt and asks the *scenario extractor* to regenerate the description. Thus, TRAFFICALIGN selectively refines recoverable scenarios and discards only semantically irrelevant ones, which helps capture the original traffic distribution of the target region.

3.3. LLM Alignment

To align the LLM's knowledge with real-world traffic distributions, TRAFFICALIGN fine-tunes the LLM using the traffic scenario descriptions synthesized from real-world driving videos. We use Llama-3.2-3B-Instruct [31] as the default generator model in TRAFFICALIGN. TRAFFICALIGN performs standard supervised fine-tuning (SFT) for the LLM with a next-token cross-entropy objective, computing loss only on assistant responses by masking user and system tokens. TRAFFICALIGN applies parameter-efficient fine-tuning (PEFT) via low-rank adaptation (LoRA) to attention and multilayer perceptron (MLP) projection layers. Training runs for 60 steps with a learning rate of $2e-4$. The aligned LLM can be prompted to generate traffic scenario descriptions consistent with real-world traffic distribution. Further details on LLM alignment, statistics of the six TRAFFICALIGN-generated alignment datasets, and regional behavior differences are provided in Supplementary Material.

4. Experiments

To comprehensively evaluate TRAFFICALIGN, we conduct three experiments. First, we evaluate the effectiveness of TRAFFICALIGN-generated scenarios in testing three autonomous driving models (Section 4.2). Second, we fine-tune these driving models on TRAFFICALIGN-generated scenarios and test them on a held-out set to evaluate TRAFFI-

Table 1. **Effectiveness of scenarios in autonomous driving model testing.** This table presents the average test results on three distinct autonomous driving models using scenarios generated by five baselines and by TRAFFICALIGN on six geographically diverse regions. CR: collision rate, RR: frequency of running red lights, SS: frequency of running stop signs, OR: average distance driven out of road, RF: route following stability, Comp: average percentage of route completion, TS: average time spent to complete the route, ACC: average acceleration, YV: average yaw velocity, LI: frequency of lane invasion, OS: overall score. ↑/↓: higher/lower values indicate more effective in testing the driving models. **Values in bold** indicate the best performance; values shaded green indicate that TRAFFICALIGN outperforms all baselines.

Method	Safety Level				Functionality Level			Etiquette Level			OS ↓
	CR ↑	RR ↑	SS ↑	OR ↑	RF ↓	Comp ↓	TS ↑	ACC ↑	YV ↑	LI ↑	
Learning-to-collide [12]	0.584	0.326	0.158	0.032	0.894	0.731	0.216	0.211	0.243	0.112	0.619
AdvSim [54]	0.586	0.300	0.160	0.025	0.891	0.745	0.261	0.203	0.245	0.127	0.620
Carla Scenario Generator [49]	0.676	0.313	0.161	0.036	0.890	0.741	0.244	0.215	0.243	0.131	0.573
Adv. Trajectory Optim. [68]	0.627	0.312	0.158	0.028	0.893	0.726	0.279	0.219	0.248	0.137	0.596
ChatScene [66]	0.825	0.191	0.142	0.035	0.861	0.537	0.213	0.706	0.533	0.228	0.481
TRAFFICALIGN (Los Angeles)	**0.933**	0.316	**0.283**	0.091	0.713	0.524	**0.491**	0.766	0.601	0.319	0.405
TRAFFICALIGN (New York City)	0.923	**0.466**	0.281	0.037	**0.700**	0.527	0.391	0.793	0.541	0.357	0.319
TRAFFICALIGN (Yellowstone)	0.909	0.456	0.221	**0.104**	0.708	**0.438**	0.298	**0.809**	0.476	0.355	**0.310**
TRAFFICALIGN (Yosemite)	0.903	0.420	0.179	0.039	0.740	0.582	0.477	0.756	**0.660**	0.218	0.457
TRAFFICALIGN (Small Towns)	0.852	0.186	0.149	0.038	0.796	0.561	0.266	0.735	0.517	0.283	0.437
TRAFFICALIGN (Switzerland)	0.919	0.379	0.201	0.041	0.723	0.514	0.383	0.761	0.514	**0.393**	0.429

CALIGN's effectiveness in improving driving model performance (Section 4.3). Third, we conduct a qualitative study to analyze how well TRAFFICALIGN-generated scenarios align with real-world traffic distributions (Section 4.4).

4.1. Experimental Setup

TRAFFICALIGN-generated traffic scenarios are represented as natural language descriptions, which are difficult to evaluate directly. To quantitatively assess their real-world usefulness, we convert these descriptions into Scenic [16] scripts. These Scenic scripts automatically configure the road network, weather, time, and the positions and behaviors of surrounding vehicles and pedestrians in the CARLA simulator [14]. In simulation experiments, the ego vehicle is controlled by a reinforcement learning-based autonomous driving model. All quantitative evaluations are performed using the SafeBench platform [60].

Simulation script conversion. To convert TRAFFICALIGN-generated traffic scenario descriptions into Scenic [16] scripts, we follow the algorithm introduced in TrafficComposer [52]. TrafficComposer uses an LLM to extract traffic information from its inputs, post-processes missing fields, and applies a rule-based algorithm to compose simulation scripts. We build on its implementation with two modifications. (i) TrafficComposer accepts multimodal inputs, whereas we use text-only (single-modal) natural language descriptions; therefore, we discard TrafficComposer's visual input processing pipeline. (ii) TrafficComposer proposes a compositional simulation script converter but does not implement it for Scenic [16], so we follow its algorithm design and implement a rule-based Scenic script converter.

Scenario generation baselines. For evaluation, we compare TRAFFICALIGN with five representative scenario generation baselines and five LLM baselines.

The strongest baseline is the state-of-the-art method, **ChatScene** [66], which prompts an LLM to generate textual descriptions of traffic scenarios and uses a predefined retrieval database to translate them into executable Scenic scripts. We also include two adversary-based methods, **Learning-to-Collide** [12] and **AdvSim** [54], which challenge driving models by perturbing the trajectories or initial configurations of surrounding vehicles and pedestrians. In addition, we include two rule-based methods, **Carla Scenario Generator** [49] and **Adversarial Trajectory Optimization** [68], which construct scenarios consistent with predefined real-world traffic rules and physical principles.

To ablate the effect of aligning LLM knowledge with real-world traffic distributions, we construct TRAFFICALIGN variants that do not perform LLM alignment. Instead, the LLM is prompted only to generate scenario descriptions for the specified location. We experiment with five state-of-the-art LLMs, GPT-5 (`gpt-5-2025-08-07`) [35], GPT-4o (`gpt-4o-2024-11-20`) [33], Claude Sonnet 4 (`claude-sonnet-4@20250514`) [4], DeepSeek-V3 (`DeepSeek-V3-0324`) [9], and Qwen3 (`Qwen3-32B-FP8`) [37, 62]. Prompt details are provided in the Supplementary Material.

Autonomous driving models. We conduct experiments using three widely adopted deep reinforcement learning (RL) algorithms to control the ego vehicle: Proximal Policy Optimization (PPO) [42], an on-policy stochastic algorithm; Soft Actor-Critic (SAC) [20], an off-policy stochastic algorithm; and Twin Delayed Deep Deterministic Policy Gradient (TD3) [17], a deterministic off-policy algorithm.

In our experiments, the observation space of the RL agents consists of four features: distance to the next way-

Table 2. **Ablation study on LLM alignment: scenario effectiveness in autonomous driving model testing.** This table presents the average test results on three distinct autonomous driving models using traffic scenarios in **Los Angeles**, generated by our TRAFFICALIGN and five LLMs without knowledge alignment. CR: collision rate, RR: frequency of running red lights, SS: frequency of running stop signs, OR: average distance driven out of road, RF: route following stability, Comp: average percentage of route completion, TS: average time spent to complete the route, ACC: average acceleration, YV: average yaw velocity, LI: frequency of lane invasion, OS: overall score, ↑/↓: higher/lower values indicate more effective in testing the driving models. **Values in bold** indicate the best performance.

Method	Safety Level				Functionality Level			Etiquette Level			OS ↓
	CR ↑	RR ↑	SS ↑	OR ↑	RF ↓	Comp ↓	TS ↑	ACC ↑	YV ↑	LI ↑	
GPT-5	0.889	0.291	0.163	0.034	0.784	0.557	0.251	0.728	0.529	0.245	0.435
GPT-4o	0.874	0.284	0.165	0.035	0.791	0.549	0.268	0.729	0.531	0.242	0.441
Claude Sonnet 4	0.894	0.243	0.187	0.037	0.786	0.539	0.259	0.714	0.541	0.241	0.438
DeepSeek-V3	0.814	0.273	0.159	0.033	0.806	0.583	0.238	0.724	0.527	0.239	0.447
Qwen3	0.804	0.278	0.153	0.035	0.796	0.571	0.207	0.719	0.522	0.219	0.450
TRAFFICALIGN	**0.933**	**0.316**	**0.283**	**0.091**	**0.713**	**0.524**	**0.491**	**0.766**	**0.601**	**0.319**	**0.405**

point, longitudinal speed, angular speed, and a detection signal for front-facing vehicles.

Quantitative Metrics. We measure the performance of driving models using 10 metrics provided by the SafeBench platform [60] across three performance levels: the Safety level (collision rate and adherence to traffic signals), the Functionality level (route adherence and completion), and the Etiquette level (smoothness of driving and lane discipline). An overall score is computed by aggregating all 10 metrics to provide a holistic assessment of performance. Our analysis primarily focuses on the collision rate and the overall score.

4.2. Effectiveness in Testing Driving Models

We evaluate the effectiveness of TRAFFICALIGN in generating challenging and safety-critical traffic scenarios for autonomous driving models. We compare TRAFFICALIGN with the state-of-the-art scenario generation method ChatScene [66], two adversary-based methods [12, 54], two rule-based methods [49, 68], and five LLM baselines.

Experiment setup. The evaluation follows the experimental settings of the state-of-the-art baseline ChatScene [66]. The evaluation pipeline first mutates each generated traffic scenario to identify challenging variants. The procedure measures scenario difficulty with a surrogate ego vehicle controlled by SAC, based on collision occurrence and the *overall score* reported from the SafeBench platform [60]. For each traffic scenario, the procedure performs 50 rounds of search and applies parameter updates every 10 steps. From the 50 variants, the two most challenging are retrained for evaluation. The scenarios are evaluated using ego vehicles trained with three driving models—PPO, SAC, and TD3. ChatScene [66] starts with 40 initial traffic scenarios. To ensure a fair comparison, TRAFFICALIGN and the five unaligned LLM baselines are evaluated under identical settings, randomly generating 40 scenarios as starting points to match ChatScene [66]. For a comprehensive evaluation, we evaluate six instances of TRAFFICALIGN, each constructed for a distinct geographic region. This quantifies the effectiveness of scenario generation methods in testing autonomous driving models.

For the two adversary-based methods, Learning-to-Collide [12] and AdvSim [54], and the two rule-based methods, Carla Scenario Generator [49] and Adversarial Trajectory Optimization [68], we adopt ChatScene's implementation.

Evaluation results. Table 1 presents the average test results of three driving models in scenarios generated by TRAFFICALIGN and five representative baselines. All TRAFFICALIGN instances across the six geographically diverse regions consistently outperform existing baseline methods on almost all metrics. Specifically, TRAFFICALIGN generates significantly more prone-to-crash traffic scenarios, achieving a 2.7%–10.8% increase in *collision rate* compared with the best-performing baseline. In terms of driving models' overall performance, TRAFFICALIGN induces a relative reduction in the average *overall score*, ranging from 5.0% to 35.6% compared with the strongest baseline. These results indicate that TRAFFICALIGN generates more prone-to-crash and challenging traffic scenarios, thereby enabling more effective testing of autonomous driving models.

An **ablation study** evaluates the effect of aligning LLM's knowledge with real-world traffic distribution. Table 2 presents the average test results of three driving models in scenarios generated by TRAFFICALIGN and five LLM baselines. Due to space limitations, we only report results from the *Los Angeles* region. TRAFFICALIGN consistently outperforms the LLM baselines across all 11 metrics, with a 3.9%–12.9% increase in *collision rate* and a 6.9%–10.0% reduction in *overall score*. These results indicate that alignment with real-world distributions enables TRAFFICALIGN to generate more challenging traffic scenarios, thereby enabling more effective autonomous driving model testing.

In conclusion, TRAFFICALIGN-generated scenarios are effective in testing autonomous driving models. The ablation study shows that aligning LLM knowledge with the real-world traffic distribution generates more challenging

Table 3. **Effectiveness of scenarios in improving autonomous driving model performance**: This table presents the average test results for three autonomous driving models after fine-tuning. The driving models are fine-tuned on a subset of scenarios generated by each method and tested on a composite held-out set aggregating test scenarios from all methods. CR: collision rate, RR: frequency of running red lights, SS: frequency of running stop signs, OR: average distance driven out of road, RF: route following stability, Comp: average percentage of route completion, TS: average time spent to complete the route, ACC: average acceleration, YV: average yaw velocity, LI: frequency of lane invasion, OS: overall score. ↑/↓: higher/lower the more effective in improving driving model performance. **Values in bold** indicate the best performance; values shaded green indicate that TRAFFICALIGN outperforms all baselines.

Location	Method	Safety Level				Functionality Level			Etiquette Level			OS ↑
		CR ↓	RR ↓	SS ↓	OR ↓	RF ↑	Comp ↑	TS ↓	ACC ↓	YV ↓	LI ↓	
Los Angeles	w/o fine-tuning	0.968	0.603	0.149	0.116	0.093	0.291	0.741	0.771	0.861	0.701	0.296
	ChatScene [66]	0.733	0.461	0.122	0.009	0.631	0.304	0.628	0.709	0.533	0.581	0.301
	TRAFFICALIGN	**0.701**	**0.388**	**0.000**	**0.000**	**0.793**	**0.598**	**0.256**	**0.613**	**0.516**	**0.498**	**0.385**
New York City	w/o fine-tuning	0.973	0.591	0.168	0.108	0.114	0.296	0.728	0.734	0.858	0.696	0.299
	ChatScene [66]	0.734	0.384	**0.000**	**0.000**	0.361	0.442	0.643	0.592	0.801	0.509	0.314
	TRAFFICALIGN	**0.710**	**0.378**	**0.000**	**0.000**	**0.800**	**0.595**	**0.258**	**0.568**	**0.501**	**0.399**	**0.366**
Yellowstone	w/o fine-tuning	0.953	0.633	0.231	0.102	0.108	0.301	0.733	0.741	0.842	0.652	0.310
	ChatScene [66]	0.768	**0.410**	0.060	**0.000**	0.753	0.528	0.413	0.671	0.591	**0.377**	0.405
	TRAFFICALIGN	**0.743**	0.419	**0.000**	**0.000**	**0.767**	**0.569**	**0.385**	**0.660**	**0.570**	0.410	**0.439**
Yosemite	w/o fine-tuning	0.965	0.609	0.204	0.133	0.103	0.305	0.731	0.730	0.854	0.663	0.304
	ChatScene [66]	0.641	0.459	**0.000**	**0.002**	0.104	0.498	0.366	0.724	0.593	0.461	0.427
	TRAFFICALIGN	**0.604**	**0.435**	**0.000**	**0.002**	**0.841**	**0.527**	**0.352**	**0.666**	**0.538**	**0.428**	**0.455**
Small Towns	w/o fine-tuning	0.956	0.582	0.201	0.109	0.142	0.331	0.671	0.734	0.833	0.602	0.311
	ChatScene [66]	0.649	0.536	**0.000**	0.031	0.653	**0.541**	0.591	0.713	0.766	**0.409**	0.401
	TRAFFICALIGN	**0.629**	**0.296**	**0.000**	**0.029**	**0.733**	0.532	**0.348**	**0.501**	**0.588**	0.426	**0.433**
Switzerland	w/o fine-tuning	0.954	0.596	0.109	0.098	0.116	0.295	0.704	0.728	0.846	0.688	0.303
	ChatScene [66]	0.653	0.431	**0.000**	**0.000**	0.682	0.392	0.461	0.639	**0.532**	**0.409**	0.389
	TRAFFICALIGN	**0.625**	**0.428**	**0.000**	**0.000**	**0.699**	**0.566**	**0.391**	**0.591**	0.533	0.412	**0.409**

scenarios. The results highlight the potential of TRAFFICALIGN to establish new benchmarks for testing the safety and robustness of autonomous driving models.

4.3. Effectiveness in Improving Driving Models

This experiment evaluates the effectiveness of TRAFFICALIGN in improving autonomous driving model performance. We fine-tune the three driving models on scenarios generated by different methods and test them on a held-out composite test set to quantify the performance improvement.

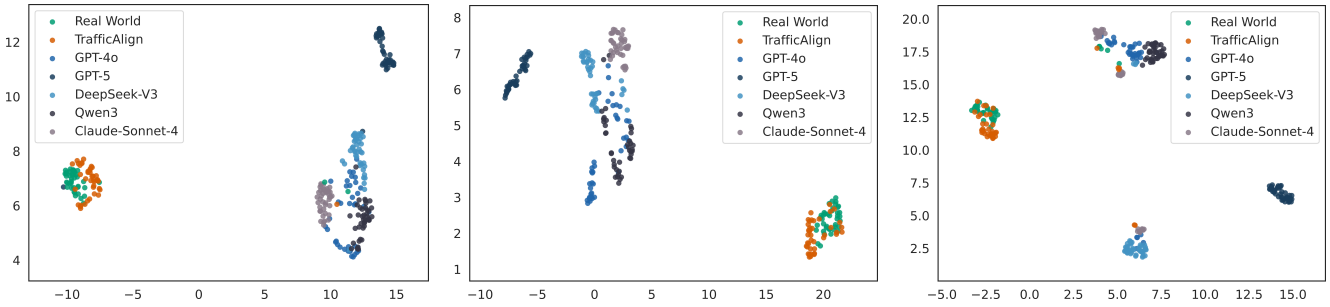
Experiment setup. For consistency, we fine-tune the same three autonomous driving models—PPO, SAC, and TD3. For each scenario generation method, we use 48 scenarios for model fine-tuning (from 40×2 scenario variants with a $3/5$ split) and evaluate on the remaining 32 ($2/5$) scenarios, aggregated across all generation methods to form a held-out composite test set. We follow the experiment settings of ChatScene [66] for model fine-tuning and evaluation. For each driving model, we initialize from the SafeBench [60] provided model checkpoint and fine-tune it for 500 epochs with a learning rate of 0.0001. We evaluate performance every 50 epochs and report the best result achieved across these evaluations. Given the high cost of agent fine-tuning, we do not include the four baselines [12, 49, 54, 68] that are significantly weaker in model testing (Section 4.2) and focus on comparison with the state-of-the-art baseline, ChatScene [66].

Evaluation Results. Table 3 presents the average test results of three autonomous driving models after fine-tuning on scenarios generated by ChatScene [66] and TRAFFICALIGN. Models fine-tuned on TRAFFICALIGN-generated scenarios consistently achieve the best performance on almost all 11 metrics across the six geographically diverse regions. Specifically, we observe a significant 21.0%–36.1% reduction in collision rate compared with the original driving model without fine-tuning, and a relative improvement in overall score of 22.4%–49.7%. Compared with the state-of-the-art baseline, ChatScene [66], TRAFFICALIGN achieves a further reduction in collision rate by 2.0%–3.7% and a further improvement in overall score by 5.1%–27.9%.

In conclusion, TRAFFICALIGN-generated scenarios are highly effective in improving autonomous driving model performance. Fine-tuning driving models on the real-world aligned scenarios generated by TRAFFICALIGN significantly improves robustness and overall performance.

4.4. Qualitative Study on Distribution Alignment

This qualitative study assesses whether the generated traffic scenarios align with the real-world traffic distribution.



(a) Los Angeles. (b) Yellowstone. (c) Small Towns in Pennsylvania, USA.

Figure 3. UMAP visualization of embeddings of traffic scenarios generated by TRAFFICALIGN, five unaligned LLM baselines, and real-world traffic scenarios.

Experiment Setup. We generate scenarios for a target region using TRAFFICALIGN and five unaligned LLM baselines and compare them with real-world scenarios extracted from driving videos. The real-world samples are excluded from the LLM alignment for TRAFFICALIGN. We embed the scenario descriptions using the Sentence Transformers `all-mpnet-base-v2` model [38, 50], then project the embeddings into two-dimensional space using Uniform Manifold Approximation and Projection (UMAP) [30] for visualization. For the best visualization, we generate 40 scenarios per method. Due to page limit, we only include the results of three regions—Los Angeles, Yellowstone, and the small town area in Pennsylvania, USA. Full results are reported in Supplementary Material.

Evaluation Results. Figure 3 visualizes scenarios generated by TRAFFICALIGN and the five unaligned LLM baselines alongside real-world cases in the embedding space. Across the three geographically diverse regions, TRAFFICALIGN-generated scenarios consistently co-locate with and overlap the *Real World* cluster. In contrast, GPT-4o, Claude-Sonnet-4, DeepSeek-V3, and Qwen3 form clusters displaced from the real-world region with barely any overlap, and GPT-5 forms an isolated cluster far from the real-world cases. In conclusion, the observed co-location and overlap indicate that TRAFFICALIGN captures the dominant modes of the real-world traffic distribution and reduces distribution shift relative to unaligned LLM baselines.

5. Limitations and Future Work

TRAFFICALIGN automatically synthesizes and validates traffic scenarios for LLM alignment from in-the-wild driving videos, but has limitations that motivate future work. First, TRAFFICALIGN uniformly samples frames from videos (Section 3.1), keeping the scenario set representative of the target region, but can potentially miss rare traffic events. Future versions can adopt distribution-aware or event-triggered sampling to better cover long-tail situations. Second, the scenario extractor operates on a single frame per scenario and depends on a multimodal LLM to perceive visual information and infer traffic dynamics. This design keeps the pipeline simple, but can lose temporal cues for understanding actor behavior. A promising future direction is to use video clips and specialized computer vision models to extract visual information while using LLMs for high-level reasoning. However, we emphasize that TRAFFICALIGN aims to propose a paradigm to automatically align LLMs with real-world traffic scenarios. Although these limitations may affect performance, they do not undermine the validity of the findings.

6. Conclusion

In this work, we introduce TRAFFICALIGN, an automated framework that synthesizes traffic scenarios based on real-world driving videos, performs data validation, and aligns LLMs with the synthesized scenarios. Our evaluation shows that traffic scenarios generated by TRAFFICALIGN are effective in autonomous driving model testing, revealing up to 10.8% more collisions on average across three driving models compared with state-of-the-art methods. Furthermore, fine-tuning these driving models with TRAFFICALIGN-generated scenarios significantly reduced collision rates by 36.1% compared with the original models, indicating TRAFFICALIGN's effectiveness in improving driving model performance. A qualitative study using traffic datasets from six geographically diverse regions shows that TRAFFICALIGN-generated scenarios align closely with the corresponding traffic distributions in these regions. Looking forward, TRAFFICALIGN can facilitate the automatic construction of traffic-safety scenario benchmarks, enabling consistent testing and targeted improvement for autonomous driving models.

Acknowledgement

We would like to thank the anonymous reviewers for their valuable feedback. This work is partly supported by the National Science Foundation under the grant IIS-2416835.

References

[1] Raja Ben Abdessalem, Annibale Panichella, Shiva Nejati, Lionel C Briand, and Thomas Stifter. Testing autonomous cars for feature interaction failures using many-objective search. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pages 143–154, 2018. 2

[2] Yasasa Abeysirigoonawardena, Florian Shkurti, and Gregory Dudek. Generating adversarial driving scenarios in high-fidelity simulators. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8271–8277, 2019. 2

[3] Aizierjiang Aiersilan. Generating traffic scenarios via in-context learning to learn better motion planner. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, pages 14539–14547. AAAI Press, 2025. 1

[4] Anthropic. System card: Claude opus 4 & claude sonnet 4. https://www-cdn.anthropic.com/6be99a52cb68eb70eb9572b4cafad13df32ed995.pdf, 2025. Accessed: 2025-10-27. 5

[5] Gerrit Bagschik, Till Menzel, and Markus Maurer. Ontology based scene creation for the development of automated vehicles. In *2018 IEEE Intelligent Vehicles Symposium (IV)*, pages 1813–1820, 2018. 2

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 3

[7] Mingfei Cheng, Yuan Zhou, and Xiaofei Xie. Behavexplor: Behavior diversity guided testing for autonomous driving systems. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 488–500, 2023. 2

[8] Kashyap Chitta, Daniel Dauner, and Andreas Geiger. Sledge: Synthesizing driving environments with generative models and rule-based traffic. In *European Conference on Computer Vision*, pages 57–74. Springer, 2024. 2

[9] DeepSeek-AI. Deepseek-v3 technical report, 2024. 5

[10] Yao Deng, Zhi Tu, Jiaohong Yao, Mengshi Zhang, Tianyi Zhang, and James Xi Zheng. TARGET: traffic rule-based test generation for autonomous driving via validated llm-guided knowledge extraction. *IEEE Trans. Software Eng.*, 51(7): 1950–1968, 2025. 1, 2, 4, 7

[11] Jeevan Devaranjan, Amlan Kar, and Sanja Fidler. Meta-sim2: Unsupervised learning of scene structure for synthetic data generation. In *European Conference on Computer Vision*, pages 715–733. Springer, 2020. 2

[12] Wenhao Ding, Baiming Chen, Minjun Xu, and Ding Zhao. Learning to collide: An adaptive safety-critical scenarios generating method. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2243–2250. IEEE, 2020. 1, 5, 6, 7

[13] Wenhao Ding, Mengdi Xu, and Ding Zhao. Cmts: A conditional multiple trajectory synthesizer for generating safety-critical driving scenarios. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4314–4321, 2020. 2

[14] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*, pages 1–16. PMLR, 2017. 5

[15] Karim Elmaaroufi, Devan Shanker, Ana Cismaru, Marcell Vazquez-Chanlatte, Alberto Sangiovanni-Vincentelli, Matei Zaharia, and Sanjit A Seshia. Scenicnl: generating probabilistic scenario programs from natural language. *arXiv preprint arXiv:2405.03709*, 2024. 1, 2

[16] Daniel J. Fremont, Edward Kim, Tommaso Dreossi, Shromona Ghosh, Xiangyu Yue, Alberto L. Sangiovanni-Vincentelli, and Sanjit A. Seshia. Scenic: a language for scenario specification and data generation. *Mach. Learn.*, 112 (10):3805–3849, 2023. 2, 5

[17] Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018. 5

[18] Alessio Gambi, Tri Huynh, and Gordon Fraser. Generating effective test cases for self-driving cars from police reports. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, page 257–267, New York, NY, USA, 2019. Association for Computing Machinery. 2

[19] An Guo, Yuan Zhou, Haoxiang Tian, Chunrong Fang, Yunjian Sun, Weisong Sun, Xinyu Gao, Anh Tuan Luu, Yang Liu, and Zhenyu Chen. Sovar: Build generalizable scenarios from accident reports for autonomous driving testing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 268–280, 2024. 2

[20] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018. 5

[21] Saffron Huang, Divya Siddarth, Liane Lovitt, Thomas I. Liao, Esin Durmus, Alex Tamkin, and Deep Ganguli. Collective constitutional AI: aligning a language model with public input. In *The 2024 ACM Conference on Fairness, Accountability, and Transparency, FAccT 2024, Rio de Janeiro, Brazil, June 3-6, 2024*, pages 1395–1417. ACM, 2024. 2

[22] Amlan Kar, Aayush Prakash, Ming-Yu Liu, Eric Cameracci, Justin Yuan, Matt Rusiniak, David Acuna, Antonio Torralba, and Sanja Fidler. Meta-sim: Learning to generate synthetic datasets. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. 2

[23] Seulbae Kim, Major Liu, Junghwan" John" Rhee, Yuseok Jeon, Yonghwi Kwon, and Chung Hwan Kim. Drivefuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1753–1767, 2022. 2

[24] Hannah Rose Kirk, Alexander Whitefield, Paul Röttger, Andrew Bean, Katerina Margatina, Juan Ciro, Rafael Mosquera,

Max Bartolo, Adina Williams, He He, Bertie Vidgen, and Scott A. Hale. The prism alignment dataset: What participatory, representative and individualised human feedback reveals about the subjective and multicultural alignment of large language models. In *Advances in Neural Information Processing Systems*, pages 105236–105344. Curran Associates, Inc., 2024. 2

[25] Moritz Klischat, Edmond Irani Liu, Fabian Holtke, and Matthias Althoff. Scenario factory: Creating safety-critical traffic scenarios for automated vehicles. In *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, pages 1–7, 2020. 2

[26] Ritchie Lee, Mykel J. Kochenderfer, Ole J. Mengshoel, Guillaume P. Brat, and Michael P. Owen. Adaptive stress testing of airborne collision avoidance systems. In *2015 IEEE/AIAA 34th Digital Avionics Systems Conference (DASC)*, pages 6C2–1–6C2–13, 2015. 2

[27] Guanpeng Li, Yiran Li, Saurabh Jha, Timothy Tsai, Michael Sullivan, Siva Kumar Sastry Hari, Zbigniew Kalbarczyk, and Ravishankar Iyer. Av-fuzzer: Finding safety violations in autonomous driving systems. In *2020 IEEE 31st international symposium on software reliability engineering (ISSRE)*, pages 25–36. IEEE, 2020. 2

[28] Sheng-Chieh Lin, Luyu Gao, Barlas Oguz, Wenhan Xiong, Jimmy Lin, Wen-tau Yih, and Xilun Chen. Flame: Factuality-aware alignment for large language models. *Advances in Neural Information Processing Systems*, 37:115588–115614, 2024. 2

[29] Yixing Luo, Xiao-Yi Zhang, Paolo Arcaini, Zhi Jin, Haiyan Zhao, Fuyuki Ishikawa, Rongxin Wu, and Tao Xie. Targeting requirements violations of autonomous driving systems by dynamic evolutionary search. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 279–291. IEEE, 2021. 2

[30] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. 8

[31] Meta. Llama-3.2-3B-Instruct. https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct, 2024. Hugging Face model card, accessed March, 2026. 4

[32] NHTSA. Standing general order on crash reporting. https://www.nhtsa.gov/laws-regulations/standing-general-order-crash-reporting, 2025. 1

[33] OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 5

[34] OpenAI. GPT-4.1 nano. https://platform.openai.com/docs/models/gpt-4.1-nano, 2025. 3

[35] OpenAI. GPT-5 system card. https://cdn.openai.com/gpt-5-system-card.pdf, 2025. 4, 5

[36] Ethan Pronovost, Meghana Reddy Ganesina, Noureldin Hendy, Zeyu Wang, Andres Morales, Kai Wang, and Nick Roy. Scenario diffusion: Controllable driving scenario generation with diffusion. In *Advances in Neural Information Processing Systems*, pages 68873–68894. Curran Associates, Inc., 2023. 2

[37] Qwen Team. Qwen3-32b-fp8. https://huggingface.co/Qwen/Qwen3-32B-FP8, 2025. FP8-quantized release; Apache-2.0 license. 5

[38] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019. 8

[39] Davis Rempe, Jonah Philion, Leonidas J Guibas, Sanja Fidler, and Or Litany. Generating useful accident-prone driving scenarios via a learned traffic prior. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17305–17315, 2022. 2

[40] Elias Rocklage, Heiko Kraft, Abdullah Karatas, and Jörg Seewig. Automated scenario generation for regression testing of autonomous vehicles. In *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, pages 476–483, 2017. 2

[41] John M. Scanlon, Kristofer D. Kusano, Tom Daniel, Christopher Alderson, Alexander Ogle, and Trent Victor. Waymo simulated driving behavior in reconstructed fatal crashes within an autonomous vehicle operating domain. *Accident Analysis & Prevention*, 163:106454, 2021. 2

[42] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 5

[43] Murray Shanahan, Kyle McDonell, and Laria Reynolds. Role-play with large language models, 2023. 3

[44] Zihao Sheng, Zilin Huang, Yansong Qu, Yue Leng, and Sikai Chen. Talk2traffic: Interactive and editable traffic scenario generation for autonomous driving with multimodal large language model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3827–3836, 2025. 1

[45] Yang Sun, Christopher M Poskitt, Jun Sun, Yuqi Chen, and Zijiang Yang. Lawbreaker: An approach for specifying traffic laws and fuzzing autonomous vehicles. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–12, 2022. 2

[46] Simon Suo, Sebastian Regalado, Sergio Casas, and Raquel Urtasun. Trafficsim: Learning to simulate realistic multi-agent behaviors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10400–10409, 2021. 2

[47] Shuhan Tan, Boris Ivanovic, Xinshuo Weng, Marco Pavone, and Philipp Krähenbühl. Language conditioned traffic generation. In *Conference on Robot Learning, CoRL 2023, 6-9 November 2023, Atlanta, GA, USA*, pages 2714–2752. PMLR, 2023. 1, 2

[48] Shuncheng Tang, Zhenya Zhang, Jixiang Zhou, Lei Lei, Yuan Zhou, and Yinxing Xue. Legend: A top-down approach to scenario generation of autonomous driving systems assisted by large language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1497–1508, 2024. 1

[49] CARLA Scenario Runner Team. Carla scenario runner. https://github.com/carla-simulator/scenario_runner, 2019. 1, 5, 6, 7

[50] SentenceTransformers Team. sentence-transformers/all-mpnet-base-v2. https://huggingface.co/sentence-transformers/all-mpnet-base-v2, 2023. Accessed: 2025-11-07. 8

[51] Haoxiang Tian, Guoquan Wu, Jiren Yan, Yan Jiang, Jun Wei, Wei Chen, Shuo Li, and Dan Ye. Generating critical test scenarios for autonomous driving systems via influential behavior patterns. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–12, 2022. 2

[52] Zhi Tu, Liangkun Niu, Wei Fan, and Tianyi Zhang. Multimodal traffic scenario generation for autonomous driving system testing. *Proc. ACM Softw. Eng.*, 2(FSE), 2025. 1, 2, 5

[53] Robin Van Der Made, Martijn Tideman, Ulrich Lages, Roman Katz, and Martin Spencer. Automated generation of virtual driving scenarios from test drive data. In *24th International Technical Conference on the Enhanced Safety of Vehicles (ESV) National Highway Traffic Safety Administration*, 2015. 2

[54] Jingkang Wang, Ava Pun, James Tu, Sivabalan Manivasagam, Abbas Sadat, Sergio Casas, Mengye Ren, and Raquel Urtasun. Advsim: Generating safety-critical scenarios for self-driving vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9909–9918, 2021. 1, 5, 6, 7

[55] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 13484–13508, 2023. 2

[56] Yuanfu Wang, Pengyu Wang, Chenyang Xi, Bo Tang, Junyi Zhu, Wenqiang Wei, Chen Chen, Chao Yang, Jingfeng Zhang, Chaochao Lu, Yijun Niu, Keming Mao, Zhiyu Li, Feiyu Xiong, Jie Hu, and Mingchuan Yang. Adversarial preference learning for robust LLM alignment. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 21865–21881. Association for Computational Linguistics, 2025.

[57] Zifeng Wang, Chun-Liang Li, Vincent Perot, Long T Le, Jin Miao, Zizhao Zhang, Chen-Yu Lee, and Tomas Pfister. Codeclm: Aligning language models with tailored synthetic data. *arXiv preprint arXiv:2404.05875*, 2024. 2

[58] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35: 24824–24837, 2022. 3

[59] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro Von Werra, Arjun Guha, and Lingming Zhang. Selfcodealign: Self-alignment for code generation. *Advances in Neural Information Processing Systems*, 37:62787–62874, 2024. 2

[60] Chejian Xu, Wenhao Ding, Weijie Lyu, Zuxin Liu, Shuai Wang, Yihan He, Hanjiang Hu, Ding Zhao, and Bo Li. Safebench: a benchmarking platform for safety evaluation of autonomous vehicles. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2022. Curran Associates Inc. 1, 5, 6, 7

[61] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. 2

[62] An Yang, Anfeng Li, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 5

[63] Zhenpei Yang, Yuning Chai, Dragomir Anguelov, Yin Zhou, Pei Sun, Dumitru Erhan, Sean Rafferty, and Henrik Kretzschmar. Surfelgan: Synthesizing realistic sensor data for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 2

[64] YouTube. Youtube. https://www.youtube.com, 2025. 3

[65] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. 2

[66] Jiawei Zhang, Chejian Xu, and Bo Li. Chatscene: Knowledge-enabled safety-critical scenario generation for autonomous vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15459–15469, 2024. 1, 2, 5, 6, 7

[67] Linrui Zhang, Zhenghao Peng, Quanyi Li, and Bolei Zhou. Cat: Closed-loop adversarial training for safe end-to-end driving. In *Proceedings of The 7th Conference on Robot Learning*, pages 2357–2372. PMLR, 2023. 2

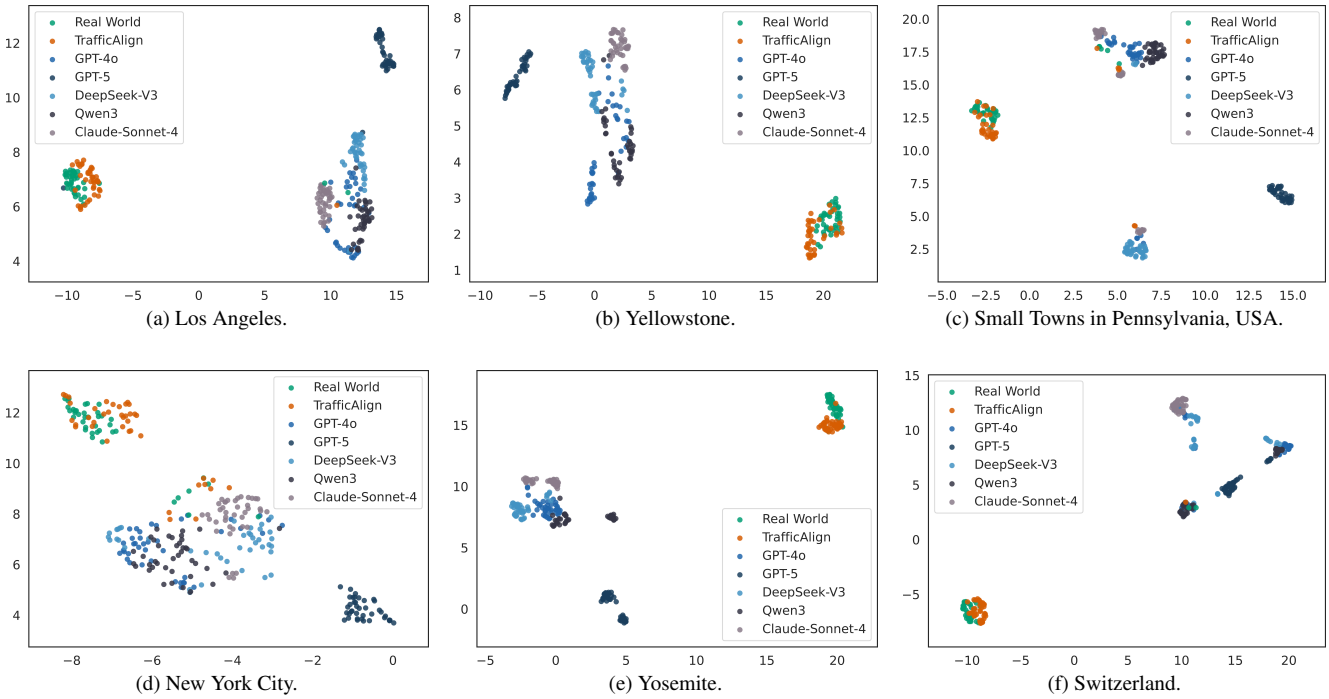
[68] Qingzhao Zhang, Shengtuo Hu, Jiachen Sun, Qi Alfred Chen, and Z Morley Mao. On adversarial robustness of trajectory prediction for autonomous vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15159–15168, 2022. 1, 5, 6, 7

TrafficAlign: Aligning Large Language Models for Traffic Scenario Generation

Supplementary Material

A. Qualitative Study on Traffic Scenario Distribution Alignment

In this section, we present qualitative results on all six geographically diverse regions considered in our study, complementing the subset of three regions reported in Section 4.4 due to page limit. For each region, we visualize the embedding-space distributions of traffic scenarios generated by TRAFFICALIGN and the five unaligned LLM baselines, alongside real-world traffic scenarios.



(a) Los Angeles. (b) Yellowstone. (c) Small Towns in Pennsylvania, USA.

(d) New York City. (e) Yosemite. (f) Switzerland.

Figure 4. UMAP visualization of embeddings of traffic scenarios generated by TRAFFICALIGN, five unaligned LLM baselines, and real-world traffic scenarios.

Across all six regions, we observe the same qualitative trend as in the Section 4.4 Figure 3): TRAFFICALIGN-generated scenarios consistently co-locate with and overlap the real-world cluster, indicating close alignment with real-world traffic distributions. In contrast, GPT-4o, Claude-Sonnet-4, DeepSeek-V3, Qwen3, and GPT-5 form clusters that are clearly displaced from the real-world region, with limited or no overlap. This consistent pattern across all datasets further supports that TRAFFICALIGN captures the dominant modes of the real-world traffic distribution and mitigates distribution shift relative to unaligned LLM baselines.

B. Ablation Study on Time Window Selection

TRAFFICALIGN uniformly samples frames from videos every 15 seconds (FPS=1/15), since the videos we collected from YouTube mainly depict everyday driving scenarios and rarely include short-horizon edge cases, making 15 seconds sufficient. However, in more intense driving videos, shorter time windows may better capture short-horizon dynamics. To examine this, we curate 10 intense driving videos from YouTube and then compare 5s vs. 15s sampling rate under the same settings as Section 4.2 and 4.3.

Table 4 shows the effectiveness of the scenarios in testing and improving autonomous driving model performances. The results show that a shorter time window can indeed catch more challenging driving scenarios and enhance the effectiveness of scenario-based testing. An interesting future direction is to adaptively detect short-horizon interactions using keyframe sampling methods instead of suing s fixed sampling interval.

Table 4. **Scenario effectiveness across time windows.** This table presents the average test results on three distinct autonomous driving models using scenarios generated from ten intense videos by TRAFFICALIGN using frame sampling rates of 5 seconds and 15 seconds. CR: collision rate, RR: frequency of running red lights, SS: frequency of running stop signs, OR: average distance driven out of road, RF: route following stability, Comp: average percentage of route completion, TS: average time spent to complete the route, ACC: average acceleration, YV: average yaw velocity, LI: frequency of lane invasion, OS: overall score. ↑/↓: higher/lower values indicate more effective in testing the driving models. **Values in bold** indicate the best performance;

(a) Testing driving models.

Method	**Safety Level**				**Functionality Level**			**Etiquette Level**			**OS** ↓
	CR ↑	RR ↑	SS ↑	OR ↑	RF ↓	Comp ↓	TS ↑	ACC ↑	YV ↑	LI ↑	
TRAFFICALIGN (5s)	**0.907**	**0.469**	0.183	**0.207**	**0.741**	0.582	**0.477**	0.756	**0.660**	0.218	**0.431**
TRAFFICALIGN (15s)	0.899	0.456	**0.221**	0.204	0.785	**0.438**	0.298	**0.809**	0.476	**0.355**	0.441

(b) Fine-tuning driving models.

Method	**Safety Level**				**Functionality Level**			**Etiquette Level**			**OS** ↑
	CR ↓	RR ↓	SS ↓	OR ↓	RF ↑	Comp ↑	TS ↓	ACC ↓	YV ↓	LI ↓	
TRAFFICALIGN (5s)	**0.766**	**0.471**	0.007	0.005	**0.826**	**0.528**	0.311	0.631	0.533	**0.513**	**0.396**
TRAFFICALIGN (15s)	0.768	0.476	**0.000**	**0.003**	0.813	0.496	**0.298**	**0.629**	**0.519**	0.524	0.391

C. Additional Details on Data Synthesis

C.1. Prompt for Traffic Scenario Synthesis

The prompt used to synthesize traffic scenarios from real-world driving videos with GPT 4.1 nano (Section 3.1) is shown in the following box.

Prompt for Traffic Scenario Synthesis from Real-World Driving Video Frames

SYSTEM

You are a traffic domain expert. Analyze the given image and extract structured information about the traffic scenario.

TASK

Analyze the given image and extract information about the traffic scenario following the steps below. Approach this task step-by-step, take your time, and don't skip steps.

Extract the basic information of the traffic scenario:

Step 1. `weather`: one of [*clear, cloudy, foggy, rainy, snowy*], or describe in your own words.

Step 2. `time`: one of [*daytime, nighttime*], or describe in your own words.

Step 3. `road_type`: one of [*straight road, intersection, t-intersection, roundabout*].

Step 4. `one-way_or_two-way`: Determine carefully from visual evidence. A road is one-way if: (a) all visible moving vehicles travel in the same direction, (b) lane markings show no center divider separating opposing flows, (c) one-way signs are visible, or (d) there is no opposing traffic lane. A road is two-way *only* if opposing traffic lanes or oncoming vehicles are clearly visible. Do NOT default to two-way; state one-way whenever opposing traffic is absent. You MUST end this step with an explicit conclusion: state exactly *"This is a one-way road."* or *"This is a two-way road."* with no other phrasing.

Step 5. `special_lane`: note any special lanes (e.g., bike lane, tram track, bus lane), or state `none`.

Step 6. `lane_number`: Count only the lanes visible in the ego vehicle's direction of travel. Use a precise integer. Estimate if unknown.

Step 7. `traffic_sign`: one of [*stop sign, speed limit sign, yield sign, yield to pedestrian sign, school zone*], or state `none`.

Step 8. `traffic_light`: one of [*green, red, yellow, broken*], or state `none`.

Step 9. `road_context`: one of [*urban, residential, highway, country road, dirt road, hill road*], or describe in your own words.

Step 10. Observe additional and detailed information of the traffic scenario:
- Whether the traffic is heavy or light. Is there a traffic jam?
- Whether there are any traffic accidents or other road hazards.
- Whether there are any road works or lane closures.
- Whether there are any emergency vehicles.
- Describe the roadside environment, including buildings, trees, and other objects.

Step 11. Observe any other information that you think is important.

Step 12. `actors`: The input image is captured by a dashboard (forward-facing) camera mounted on the ego vehicle. This means ONLY actors in FRONT of or BESIDE the ego vehicle are visible. Do NOT describe any actor as being behind, left behind, or right behind the ego vehicle.
Before listing actors, scan the full image systematically: near field (close to the ego vehicle), mid field, and far field; left side, center, and right side. Include ALL actors that are present in the image, even if they are partially visible or at the edge of the frame. Do NOT omit any actor that can be seen. At the same time, do NOT invent or assume actors that have no visual evidence in the image.
For each detected actor (e.g., sedan, bus, pedestrian, bicycle, etc.), record the following attributes. Remember to also describe the ego vehicle.
- `type`: The classification of the actor (e.g., sedan, SUV, truck, pedestrian, bicycle, etc.).
- `current_behavior`: The actor's present action or intent (e.g., move forward, turn left, stopped, yield).

- `speed`: The current speed of the actor as an integer in km/h. You MUST provide a numeric estimate — never use vague descriptions such as "slow", "fast", "moderate", or "unknown". If the exact speed cannot be determined, estimate it based on context:
  - stopped vehicle ⇒ `0 km/h`
  - city vehicle moving normally ⇒ `30--50 km/h`
  - highway vehicle ⇒ `80--120 km/h`
- `position_target` and `position_relation`: Relative position to the ego vehicle. The valid values and their precise definitions are:
  - `front`: the actor is in the *same lane* as the ego vehicle and is ahead.
  - `left front`: the actor is in a lane to the *left* of the ego vehicle AND is ahead longitudinally.
  - `right front`: the actor is in a lane to the *right* of the ego vehicle AND is ahead longitudinally.
  - `left`: the actor is in a lane to the *left* at roughly the same longitudinal position (side-by-side).
  - `right`: the actor is in a lane to the *right* at roughly the same longitudinal position (side-by-side).

  The relations `behind`, `left behind`, and `right behind` must NOT be used. Left and right are always from the ego vehicle driver's perspective (same as left/right in the image). To assign the correct relation, first determine the actor's lane relative to the ego vehicle (same / left / right), then determine whether the actor is ahead or side-by-side.
- `lane_index`: An integer for the lane within the road network. The leftmost lane in the ego vehicle's direction is index `1`; the leftmost lane in the opposite direction is `-1`.
  - E.g., "parked in the rightmost lane" with `lane_number: 3` ⇒ `lane_index: 3`.
  - E.g., "moving forward in the leftmost lane" with `lane_number: 2` ⇒ `lane_index: 1`.

Step 13. Generate a concise, technical description of the traffic scenario that:
- Uses precise traffic engineering terminology, active voice, and present tense.
- Uses assertive language (*"is"/"are"* instead of *"appears"/"seems"*).
- Avoids image-related terms (e.g., "shows", "displays", "depicts").
- Uses precise numbers (e.g., "10 sedans" instead of "many").
- Describes road network first (road context, road type, one-way/two-way, special lanes, lane number, traffic sign, traffic light), then environment (weather, time of day), then additional details from Steps 10–11.
- Ends with an **Actors** section (bold title on a new line) listing each actor type with its precise count.

OUTPUT FORMAT

<ANALYSIS>
[Your chain of thoughts following the step-by-step instructions provided in step 1 to step 12]
</ANALYSIS>

<TEXT>
[Insert paragraph description here]
Actors
[Insert paragraph describing the actors here]
</TEXT>

IN CONTEXT LEARNING EXAMPLE

Example input image:



Example output:
<ANALYSIS>
Step 1. weather: clear
Step 2. time: daytime
Step 3. road type: straight road
Step 4. one-way or two-way: two-way
Step 5. special lane: None
Step 6. lane number: 6
Step 7. traffic sign: no traffic sign is observed
Step 8. traffic light: The traffic light is current green
Step 9. road context: urban, residential and commercial mixed-use area
Step 10. the traffic is light; there are no traffic accidents; there are no road works or lane closures; there are no special vehicles such as emergency vehicles
Step 11. No special components are worth noticing
Step 12. actors: Several types of actors are observed. The ego vehicle travels forward at about 38 km/h. A bus is in front of the ego vehicle moving at around 35 km/h. Approximately 10 sedans are parked along the street in the ego direction, and 7 sedans are parked on the rightmost lane in the opposite direction. A truck is driving toward the ego vehicle in the central oncoming lane at about 40 km/h.
</ANALYSIS>

<TEXT>
The traffic scenario occurs on an urban, two-way road with a standard lane configuration, three lanes per direction. The road is marked with one solid yellow centerline, indicating no overtaking, and has no speed limit sign present. A traffic light is visible and currently displays green, enabling vehicle passage in the direction of travel. The environment is during daytime with clear weather and bright conditions. The road is bordered by residential and commercial buildings, with trees lining both sides of the street, and sidewalks are visible along the curb. No traffic jams or accidents are evident, and there are no ongoing road works or lane closures. No emergency vehicles are present. The roadside vegetation, including trees and shrubbery, appears well-maintained, and the overall environment is calm. Traffic flow is smooth with no signs of congestion or hazards, and pedestrians are not observed in the scene.
****Actors****
The ego vehicle is traveling forward at about 38 km/h. There is a bus driving in front of the ego vehicle at around 35 km/h. There are approximately 10 sedans parked along the sides of the street in the ego vehicle's direction and 7 parked sedans on the rightmost lane in the opposite direction. There is a truck driving in the opposite direction in the most central lane at about 40 km/h.
</TEXT>

Input image:
[Insert the input image here.]

C.2. Natural Language Description Examples

This section provides two example natural language scenario descriptions (Section 3.1).

Natural Language Scenario Description Example 1

The scenario unfolds on a straight, two-lane urban roadway with a single direction of travel and no special-purpose lanes. Conditions are daytime with clear skies, providing good visibility. The surroundings consist of commercial buildings, sidewalks, and typical city infrastructure, including several tall structures. Traffic signals are green, permitting continuous vehicle flow, and there are no signs of congestion, incidents, or construction work. The road surface is unobstructed, contributing to smooth travel conditions. No emergency vehicles or hazardous elements are detected, and lane availability is unrestricted, indicating all lanes remain open for normal use.
****Actors****
The ego vehicle occupies the right lane, proceeding forward at 30 km/h. In the left lane, a bus travels straight at roughly 25 km/h. Two additional vehicles are present: a car ahead of the ego vehicle and an SUV positioned directly ahead of the bus, each moving at about 30 km/h.

Natural Language Scenario Description Example 2

The scene depicts a four-lane, straight urban roadway with single-direction traffic during bright daylight with clear weather beneath an overcast sky. Trees and high-rise buildings line both sides of the street. No traffic lights regulate movement. At the approaching intersection, a "Stop" sign controls vehicle behavior. The roadside environment consists of sidewalks, trees, and dense urban structures. Traffic is partially free-flowing with minimal congestion or hazards, and several vehicles are already in motion.
****Actors****
The ego vehicle is positioned in the second lane from the right, traveling forward at approximately 35 km/h. Seven pedestrians are crossing at the marked crosswalk. Two taxis are moving in the outer-right lane at roughly 30 km/h, while around ten stationary vehicles are parked or stopped along the right roadside. An unoccupied bus is positioned along the right-side curb, remaining stationary throughout the scene.

D. Additional Details on Data Validation

D.1. Details of the Domain-Specific Language

For efficient data validation (Section 3.2), we adopt the existing domain-specific language (DSL) design proposed by TARGET [10]. Building on the original grammar, we extend the DSL with an `actor_group` element that groups actors located in close proximity and a `lane_index` attribute that specifies lane-level positions. These extensions enable a concise and precise representation of scenes with many participants. As defined in Figure 5, the DSL follows a context-free grammar, comprising three sections: *Environment*, *Road Network*, and *Actors*.

- ***Environment*** encodes time of day and weather conditions.
- ***Road network*** encodes road type, lane configurations, and traffic signal settings.
- ***Actors*** encode the number, type, position, lane occupancy, and behaviors (e.g., go straight, turn left, etc.) of vehicles and pedestrians grouped by positions.

Scenario	::= *Environment*; *Road_network*; *Actors*
Environment	::= *weather*; *time*
weather	::= `rainy` | `foggy` | `snowy` | `wet` | `...`
time	::= `daytime` | `nighttime`
Road_network	::= *road_type*; *traffic_signals*; *lane_number*
road_type	::= `intersection` | `roundabout` | `...`
traffic_signals	::= *traffic_signs*, *traffic_light*
traffic_signs	::= ϵ | *traffic_sign*; *traffic_signs*
traffic_sign	::= `stop_sign` | `speed_limit_sign` | `...`
traffic_light	::= ϵ |`red_light` | `green_light`
lane_number	::= `0` | `1` | `2` | `3` | `...`
Actors	::= *ego_vehicle*; *npc_actors*
ego_vehicle	::= *behavior*; *position*
npc_actors	::= ϵ | *actor_group*; *npc_actors*
actor_group	::= *actor_number*; *actor_type*; *behavior*; *position*
actor_type	::= `car` | `truck` | `pedestrian` | `...`
behavior	::= `go_forward` | `turn_left` | `...`
position	::= *reference_point*; *relative_position*; *lane_index*
reference_point	::= `ego_vehicle` | *road_type* | *traffic_signals*
relative_position	::= `front` | `behind` | `left` | `on` | `...`
lane_index	::= `0` | `1` | `2` | `3` | `...`

Figure 5. The grammar of the traffic scenario DSL [10].

D.2. Prompt for Natural Language to DSL Translation

The prompt used to translate TRAFFICALIGN-synthesized traffic scenario natural language descriptions to the driving videos with GPT 5 (Section 3.1) is shown in the following box.

Prompt for Translating Natural Language Description into DSL Representation

SYSTEM

You are a traffic domain expert familiar with domain-specific languages (DSL).

TASK

Your task is to extract the information about a traffic scenario from a textual description and represent it as a DSL in YAML format. Take your time to solve this task step by step.

Steps 1–8: Extract the basic information of the traffic scenario:

Step 1. `weather`: one of [*clear, cloudy, foggy, rainy, snowy*]. Don't infer. If the natural language description does not clearly state this, use `unspecified`.

Step 2. `time`: one of [*daytime, nighttime*]. Don't infer. If the natural language description does not clearly state this, use `unspecified`.

Step 3. `road_type`: one of [*straight road, curved road, intersection, t-intersection, roundabout*]; include one-way/two-way and any special lanes (e.g., bike lane, tram track). Don't infer. If the natural language description does not clearly state this, use `unspecified`.

Step 4. `lane_number`: The number of lanes *per direction* for a two-way road. Use a precise integer. Don't infer. If not clearly stated, use `unspecified`.
- E.g., a two-way road with two lanes in each direction ⇒ `lane_number: 2`.
- E.g., a two-way road with two lanes total ⇒ `lane_number: 1`.

Step 5. `one-way_or_two-way`: one of [*one-way, two-way*]. Don't infer. If not clearly stated, use `unspecified`.

Step 6. `traffic_sign`: one of [*stop sign, speed limit sign, yield sign, yield to pedestrian sign, school zone*], or `none` if no sign is present. Don't infer. If not clearly stated, use `unspecified`.

Step 7. `traffic_light`: one of [*green, red, yellow, broken*], or `none` if no traffic light is present. Don't infer. If not clearly stated, use `unspecified`.

Step 8. `road_context`: one of [*urban, residential, highway, country road, dirt road, hill road*]. Don't infer. If not clearly stated, use `unspecified`.

Step 9. `actors`: For each actor group (e.g., car, bus, pedestrian, bicycle), extract the following attributes:
- `type`: The classification of the actor (e.g., sedan, SUV, truck, pedestrian, cyclist). Don't infer. If not clearly stated, use `unspecified`.
- `number`: The number of actors in this group. If the description states an approximate count using hedging words (e.g., "around 10", "approximately 5", "about 3", "roughly 8"), extract the integer value — do *not* treat hedging language as `unspecified`. Only use `unspecified` when no numeric information is given at all (e.g., "several", "a few", "many").
- `behavior`: The actor's present action or intent (e.g., move forward, turn left, stop, yield). Don't infer. If the natural language description does not clearly state this, use "unspecified". Use "parked" when the actor is described as parked (e.g., "parked along the roadside", "parked by the curb") — do not use "stopped" for parked actors. Use "stopped" only for actors that have temporarily halted while in traffic (e.g., waiting at a red light). Treat "stationary or moving slowly" as `stopped`.
- `speed`: The current speed in km/h. If an approximate speed is given using hedging words (e.g., "around 30 km/h", "approximately 50 km/h", "roughly 60 km/h"), extract the integer value. Use `unspecified` when: (a) no numeric speed information is given, or (b) speed is described only with a qualitative adjective (e.g., "slow", "fast", "moderate", "crawling") with no accompanying number.
- `position_target` and `position_relation`: Relative position to the ego vehicle. If impossible to infer, use `unspecified`.

  - E.g., "directly ahead of the ego vehicle in the same lane" ⇒ `position_target: ego_vehicle; position_relation: front`.
  - E.g., "to the left rear of the ego vehicle in an adjacent lane" ⇒ `position_target: ego_vehicle; position_relation: left behind`.
  - E.g., "in the opposite direction driving towards the ego vehicle" ⇒ `position_target: ego_vehicle; position_relation: front`.
  - E.g., ego vehicle is in the leftmost lane and another actor is parked by the curbside ⇒ `position_target: ego_vehicle; position_relation: right`.
- `lane_index`: An integer for the lane within the road network. The leftmost lane in the ego vehicle's direction is index `1`; the leftmost lane in the opposite direction is `-1`. Use `unspecified` if not determinable.
  - E.g., "parked in the rightmost lane" with `lane_number: 3` ⇒ `lane_index: 3`.
  - E.g., "moving forward in the leftmost lane" with `lane_number: 2` ⇒ `lane_index: 1`.
  - E.g., "parked by the curbside in the opposite direction" with `lane_number: 2` ⇒ `lane_index: -2`.

Step 10. Based on the information extracted in Steps 1–9, generate a structured YAML block. Wrap this with `<YAML>...</YAML>` as shown below.

```
<YAML>
environment:
    weather: clear  # one of: clear, cloudy, foggy, rainy, snowy, unspecified
    time: daytime   # one of: daytime, nighttime, unspecified
road_network:
    road_type: straight road  # one of: straight road, curved road,
                              #   intersection, t-intersection, roundabout,
                              #   unspecified
    road_context: urban  # one of: urban, residential, highway, country road,
                         #   dirt road, hill road, unspecified
    traffic_sign: none   # one of: stop sign, speed limit sign, yield sign,
                         #   yield to pedestrian sign, school zone, none,
                         #   unspecified
    traffic_light: none  # one of: green, red, yellow, broken, none, unspecified
    lane_number: 1       # integer or unspecified
    one-way_or_two-way: two-way  # one of: one-way, two-way, unspecified
actors:
    ego_vehicle:
        type: none           # always none for ego_vehicle
        behavior: move forward  # e.g.: move forward, stopped, turn left,
                                #   turn right, merge left, merge right,
                                #   unspecified
        speed: 30 km/h       # integer in km/h, or unspecified
        position_target: unspecified
        position_relation: unspecified  # one of: front, left, right, behind,
                                        #   left front, left behind, right
                                        ↪  front,
                                        #   right behind, unspecified
        lane_index: 1        # integer or unspecified
    other_actor_1:
        type: sedan          # e.g.: sedan, truck, tram, van, car, SUV, bus,
                             #   pedestrian, bicycle
        number: 1            # integer or unspecified
        behavior: move forward
        speed: 30 km/h       # integer in km/h, or unspecified
        position_target: ego_vehicle  # NOTE: always ego_vehicle for all
                                      #   actors other than ego_vehicle
        position_relation: front
        lane_index: 1
    ... (other actors if any) ...
</YAML>
```

IN CONTEXT LEARNING EXAMPLE

Example input 1:

The traffic scenario takes place on a two-lane, two-way urban street with no special lanes and no visible traffic signs or signals. The environment is during daytime with cloudy weather. The roadway is lined with commercial buildings and sidewalks populated by pedestrians and cyclists. Traffic is moderate, with vehicles moving smoothly and no apparent congestion, accidents, or hazards. No roadworks or lane closures are present. No emergency vehicles are observed.

Actors

The ego vehicle travels at about 28 km/h. Ahead of it, a yellow sedan moves forward at approximately 30 km/h, followed by a black SUV traveling at around 32 km/h. In the opposite lane, around four cars drive toward the ego at roughly 35 km/h.

Example output 1:

```
<YAML>
environment:
    time: daytime
    weather: cloudy
road_network:
    lane_number: 1
    one-way_or_two-way: two-way
    road_type: straight road
    traffic_light: none
    traffic_sign: none
    road_context: urban
actors:
    ego_vehicle:
        type: none
        behavior: move forward
        lane_index: 1
        position_relation: unspecified
        position_target: unspecified
        speed: 28 km/h
    vehicle_in_front_1:
        type: sedan
        number: 1
        behavior: move forward
        lane_index: 1
        position_relation: front
        position_target: ego_vehicle
        speed: 30 km/h
    vehicle_in_front_2:
        type: SUV
        number: 1
        behavior: move forward
        lane_index: 1
        position_relation: front
        position_target: ego_vehicle
        speed: 32 km/h
    vehicles_opposite_direction:
        type: car
        number: 4
        behavior: move forward
        lane_index: -1
        position_relation: front
        position_target: ego_vehicle
        speed: 35 km/h
</YAML>
```

Example input 2:
The traffic scenario takes place on a rural, two-way curved road within mountainous terrain. The road has one lane in each direction, two lanes in total. No traffic signs or lights are visible in the scene. The environment features mostly clear daytime weather, providing good visibility of the surrounding rugged landscape. Road work is in progress, as shown by a worker standing ahead in the ego vehicle's direction. Traffic is light, with no congestion, accidents, or emergency vehicles present. The roadside is characterized by rocky slopes and sparse trees, with wide open areas extending along both sides of the roadway.
Actors

The ego vehicle is moving forward at about 45 km/h. In the opposite lane, two construction trucks are traveling extremely slowly at around 3 km/h. Ahead of the ego vehicle, a road worker is standing still in the lane, remaining stationary at 0 km/h.

Example output 2:

```
<YAML>
environment:
    time: daytime
    weather: clear
road_network:
    lane_number: 1
    one-way_or_two-way: two-way
    road_type: curved road
    traffic_light: none
    traffic_sign: none
    road_context: country road
actors:
    ego_vehicle:
        type: none
        behavior: move forward
        lane_index: 1
        position_relation: unspecified
        position_target: unspecified
        speed: 45 km/h
    construction_vehicles:
        type: construction vehicle
        number: 2
        behavior: move forward
        lane_index: -1
        position_relation: front
        position_target: ego_vehicle
        speed: 3 km/h
    traffic_worker:
        type: construction worker
        number: 1
        behavior: stopped
        lane_index: 1
        position_relation: front
        position_target: ego_vehicle
        speed: 0 km/h
</YAML>
```

Input natural language description:

[Insert the input natural language description here.]

D.3. Effectiveness of DSL Validation

We conduct a preliminary comparison to compare driving model testing and fine-tuning performance on scenarios synthesized with and without the DSL component.

As shown in Table 5, incorporating DSL validation is associated with improves scenario quality across both evaluation settings. For autonomous driving model testing experiments, scenarios generated with DSL validation achieve a lower overall score (0.319 vs. 0.408) and a higher collision rate (0.925 vs. 0.873). A similar trend is also observed when using the generated scenarios in fine-tuning autonomous models, where DSL validation improves the overall score from 0.369 to 0.402 and lowers the collision rate from 0.745 to 0.691.

Table 5. **Driving model testing & fine-tuning experiments.** This table presents the average test results on three distinct autonomous driving models using scenarios generated with and without the DSL component. CR: collision rate, RR: frequency of running red lights, SS: frequency of running stop signs, OR: average distance driven out of road, RF: route following stability, Comp: average percentage of route completion, TS: average time spent to complete the route, ACC: average acceleration, YV: average yaw velocity, LI: frequency of lane invasion, OS: overall score. ↑/↓: higher/lower values indicate more effective in testing the driving models. **Values in bold** indicate the best performance;

(a) Testing driving models.

Method	**Safety Level**				**Functionality Level**			**Etiquette Level**			**OS** ↓
	CR ↑	RR ↑	SS ↑	OR ↑	RF ↓	Comp ↓	TS ↑	ACC ↑	YV ↑	LI ↑	
w/o DSL Validation	0.873	0.286	0.219	0.038	0.718	**0.496**	0.381	0.659	0.488	0.219	0.408
with DSL Validation	**0.925**	**0.461**	**0.273**	**0.041**	**0.699**	0.527	**0.391**	**0.793**	**0.541**	**0.357**	**0.319**

(b) Fine-tuning driving models.

Method	**Safety Level**				**Functionality Level**			**Etiquette Level**			**OS** ↑
	CR ↓	RR ↓	SS ↓	OR ↓	RF ↑	Comp ↑	TS ↓	ACC ↓	YV ↓	LI ↓	
w/o DSL Validation	0.745	**0.318**	0.010	0.000	0.749	0.533	0.312	0.574	0.501	0.418	0.369
with DSL Validation	**0.691**	0.366	**0.000**	0.000	**0.814**	**0.593**	**0.211**	**0.553**	**0.496**	**0.381**	**0.402**

D.4. Preliminary Evaluation of DSL Validation Accuracy

We conduct a preliminary evaluation to assess the accuracy of DSL validation and to examine the extent to which its outputs are affected by LLM hallucinations. Specifically, we sample 120 natural language scenario descriptions, manually annotate the ground-truth DSL, and compare the DSL outputs generated by the DSL converter against these annotations. We find that 13 out of 120 files contain hallucinations in the generated DSL, indicating that the DSL converter is affected by LLM hallucinations. Nevertheless, the error rate is relatively limited, and the hallucinations are concentrated in a small number of fields. The most frequent hallucination is `lane_number`, which appears in 9 files, followed by `one-way or two-way`, which appears in 4 files. We also observe one file each with a hallucination in `lane_index` and the actor `type`.

Overall, these results indicate that hallucinations are present but not pervasive in DSL validation on this preliminary benchmark.

E. Additional Details on LLM Alignment

E.1. Implementation Details for LLM Alignment

We implement the fine-tuning pipeline using the Unsloth framework to leverage 4-bit quantization, enabling memory-efficient training of the Llama-3.2-3B-Instruct model on a single NVIDIA Tesla T4 GPU. The model is trained with a maximum sequence length of $2,048$ tokens. For Parameter-Efficient Fine-Tuning (PEFT), we configure Low-Rank Adaptation (LoRA) with a rank $r = 16$ and a scaling factor $\alpha = 16$. We apply LoRA adapters to all linear projection layers, including the query (`q_proj`), key (`k_proj`), value (`v_proj`), output (`o_proj`), and the MLP layers (`gate_proj`, `up_proj`, `down_proj`), with no dropout. Optimization is performed using the 8-bit AdamW optimizer with a weight decay of 0.01. We utilize a linear learning rate scheduler with 5 warmup steps, targeting a peak learning rate of $2e-4$. The training process uses a per-device batch size of 2 with 4 gradient accumulation steps. To ensure the model learns strictly from the desired output distributions, we employ a response-only masking strategy, where loss is calculated exclusively on the assistant's generated tokens, masking the system instructions and user prompts.

E.2. Overview of the Six LLM Alignment Datasets

Table 6 summarizes the composition of the six LLM alignment datasets used in our study across diverse geographic locations.

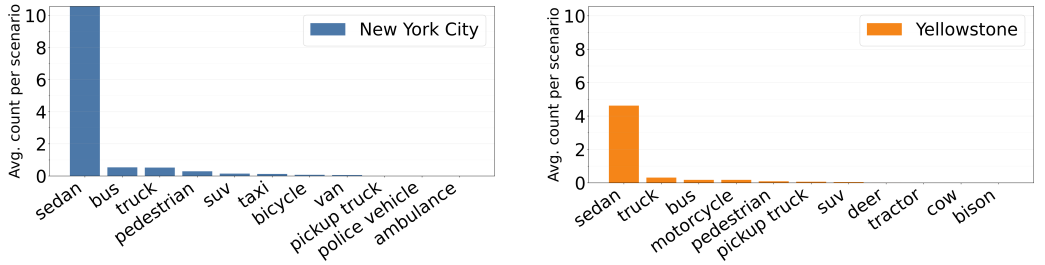
Table 6. Summary statistics of the six LLM alignment datasets across diverse geographical locations.

Location	**# Videos**	**Avg. # Actors**
Los Angeles	82	13.8
New York City	84	13.3
Yellowstone	14	2.5
Yosemite	20	2.3
Small Towns in PA	15	3.2
Switzerland	46	11.1

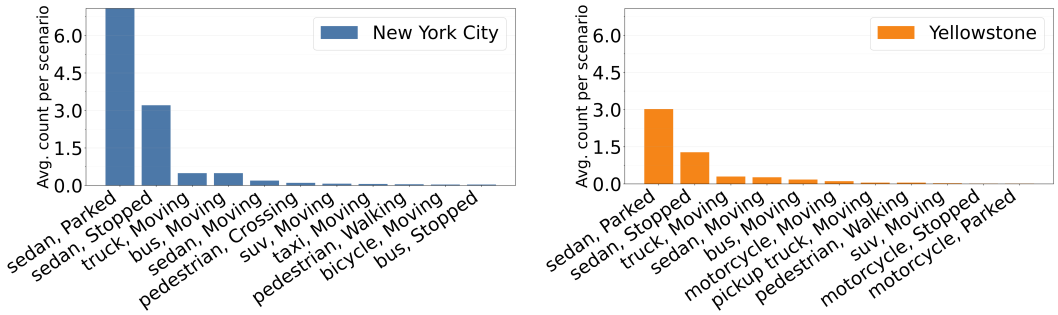
Across the six alignment datasets, the average number of actors clearly tracks the urban–rural divide. The three urban datasets, Los Angeles, New York City, and Switzerland, consistently exhibit dense interactions, with 13.8, 13.3, and 11.1 actors per scenario, respectively. In contrast, the rural and small-town datasets (Yellowstone, Yosemite, and Small Towns in PA) are much sparser, with only 2.3–3.2 actors on average.

E.3. Behavior Difference Across Geographic Regions

We analyze generated scenarios from a metropolitan (New York City) and a rural (Yellowstone) dataset. As shown in Figure 6, New York City is more crowded, with frequent pedestrian crossings and more parked sedans, while Yellowstone is sparser and features rural-only actors (e.g., deer, cows, bison, tractors) absent in New York City.



(a) Actor type distributions.



(b) Actor type-behavior distributions.

Figure 6. Behavioral differences across geographic regions.

E.4. Human Study on Scenario Semantic Fidelity

To evaluate whether TRAFFICALIGN-generated scenarios are semantically consistent with real-world traffic scenarios, we conduct a qualitative study in Section 4.4 and Section A. In this section, we further conduct a human study to assess the semantic fidelity of the generated scenarios. We sample 100 scenarios (50 New York City, 50 Yellowstone) and ask 6 graduate students to independently rate the semantic fidelity of each scenario on a 1–5 scale, measuring "how likely the traffic scenario is to occur in the specified location". The human study reports mean scores of 4.553 (New York City) and 4.627 (Yellowstone). The results show that TRAFFICALIGN generates semantically faithful, region-consistent traffic scenarios.

E.5. TRAFFICALIGN Prompt for Real-World Aligned Traffic Scenario Generation

This section presents the complete prompt used in our experiments to query the aligned LLMs introduced in Section 3.3.

Prompt for Aligned LLMs to Generate Traffic Scenarios

SYSTEM

You are a traffic domain expert.

TASK

Design a traffic scenario in real life and provide a detailed description including the environment (weather, time of day), road network (road type, road context, lane number, one-way or two-way). Then design the position, behavior and speed of the ego vehicle and details of other actors, specifying actor type, behavior, speed, position, lane index, and any interactions with the ego vehicle. Wrap the generated textual description in <TEXT>...</TEXT> tags.

F. Unaligned LLM Baseline Prompts

The prompt used to generate traffic scenarios using unaligned LLM baselines in the ablation study in Section 4.1 is shown in the following box.

Prompt for Unaligned LLM baselines to Generate Traffic Scenarios

SYSTEM

You are a traffic domain expert. Your task is to design a traffic scenario for testing autonomous driving systems.

TASK

Design a traffic scenario take place in **[Insert the region name here]**. Approach this task step-by-step, take your time, and don't skip steps.

Design the basic configurations of the traffic scenario:

Step 1. weather: one of [clear, cloudy, foggy, rainy, snowy], or describe in your own words

Step 2. time: one of [daytime, nighttime], or describe in your own words

Step 3. road type: one of [straight road, intersection, t-intersection, roundabout]

Step 4. one-way or two-way: one of [one-way, two-way]

Step 5. special lane: if there are any special lanes (e.g., bike lane, tram track, bus lane)

Step 6. lane number: use an integer.

Step 7. traffic sign: one of [stop sign, speed limit sign, yield sign, yield to pedestrian sign, school zone], or state "none"

Step 8. traffic light: one of [green, red, yellow, broken], or state "none"

Step 9. road context: one of [urban, residential, highway, country road, dirt road, hill road], or describe in your own words

Step 10. Design additional and detailed information of the traffic scenario:

- Whether the traffic is heavy or light; Is there a traffic jam?
- Whether there are any traffic accidents or other road hazards
- Whether there are any road works or lane closures
- Whether there are any emergency vehicles
- Describe the environment of the roadside, including the buildings, trees, and other objects.

Step 11. Consider any other information that you think is important.

Step 12. actors: For each detected actor (e.g., car, bus, pedestrian, bicycle), record the following attributes:

- type: The classification of the actor (e.g., sedan, SUV, truck, pedestrian, cyclist)
- current behavior: The actor's present action or intent (e.g., move forward, turn left, stop, yield)
- speed: The current speed of the actor, measured in kilometers per hour (km/h)
- position: The actor's location, which should be described in two ways: (1) relative position to the ego vehicle using position target and position relation, and (2) the absolute position in the road network using lane index
- position target and position relation: Relative position to the ego vehicle. Specify the actor's position concerning the ego vehicle.
  - For example, if another actor is "directly ahead of the ego vehicle in the same lane", position target is ego vehicle; position relation is front.
  - Another example, if another actor is "to the left rear of the ego vehicle in an adjacent lane", position target is ego vehicle; position relation is left behind.
- lane index: an integer stating the lane index within the road network. The lane index of the leftmost lane in the direction of the ego vehicle is 1. The lane index of the leftmost lane in the opposite direction of the ego vehicle is -1.
  - For example, if an actor is "parked in the rightmost lane", and the lane number is 3, then the lane index is 3.
  - Another example, if an actor is "moving forward in the leftmost lane" and the lane number is 2, then the lane index is 1.

Generate a concise, technical description of the traffic scenario that:
- Use precise traffic engineering terminology.
- Use languages to describe a traffic scenario rather than a image.
- Use active voice and present tense.
- Avoid subjective observations.
- Use assertive language (“is” / “are” instead of “appears” / “seems” / “possibly”).
- Avoid image-related terms (e.g., “shows”, “displays”, “depicts”).
- Use precise numbers (e.g., “10 cars” instead of “many cars” / “multiple cars”).
- Maintain all key information about the road network, the environment, and actors.
- Describe the road network first, then the environment, then the actors.
- For the road network, first describe the road context and the road type, one-way / two-way, and any special lanes (e.g., bike lane, tram track). Then describe the lane number, the traffic sign, and the traffic light.
- For the environment, describe the weather and time of day.
- Describe the additional and detailed information observed in step 10 and step 11.
- At the end, add the descriptions of the actors. For the actors, describe the type and count of each type of actor. Remember to include the precise number (in digits) of actors.
- Treat the actor section differently and start this section with a separate title **Actors** in a new line.

Start your generated description with <TEXT> and end with </TEXT>

OUTPUT FORMAT

<ANALYSIS>
[Your chain of thoughts following the step-by-step instructions provided in step 1 to step 12]
</ANALYSIS>
<TEXT>
[Insert paragraph description here]
Actors
[Insert paragraph describing the actors here]
</TEXT>